

# BIG DATA 07

*Practical Assignment Portfolio*

## ASSIGNMENT 1

Real-Time Data Streaming Pipeline

*Apache Kafka + InfluxDB + Grafana*

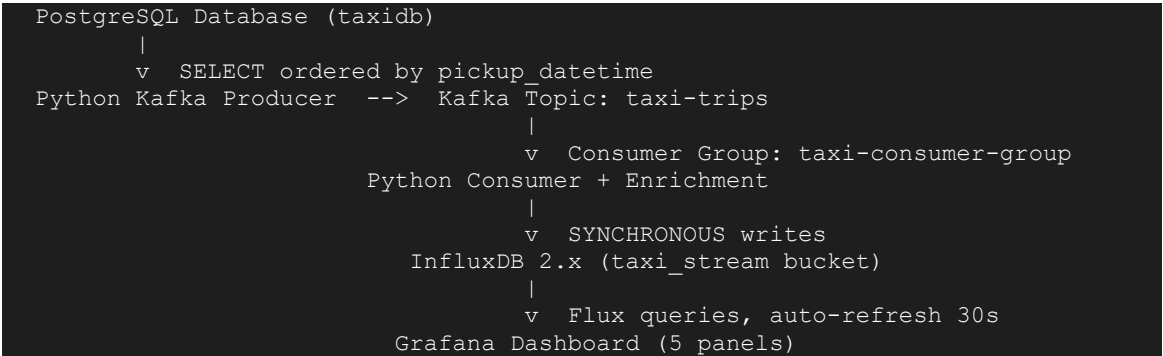
|                 |                                    |
|-----------------|------------------------------------|
| Student Name    | Hayagriva Boodhoo                  |
| Student ID      | 2025_20379_6980                    |
| Cohort          | BDA 7                              |
| Dataset Used    | NYC Yellow Taxi Trips (PostgreSQL) |
| Submission Date | 25 May 2026                        |

*Technologies Used: Apache Kafka | PostgreSQL | InfluxDB 2.x | Grafana | Python | Docker*

# 1. Assignment Overview

This assignment implements a complete real-time data streaming pipeline using industry-standard big data technologies. A Python producer reads NYC taxi trip records from a PostgreSQL database and publishes them to Apache Kafka. A consumer reads from the Kafka topic, computes four enrichment fields, and writes the enriched records to InfluxDB. A Grafana dashboard with five panels visualises the live stream with auto-refresh.

## Pipeline Architecture



# 2. Dataset Description

The dataset used is the NYC Yellow Taxi Trips dataset, loaded into a PostgreSQL relational database container (taxidb). The dataset consists of 25,000 synthetic taxi trip records generated using PostgreSQL's generate\_series() function with realistic NYC bounding box coordinates, fare distributions, and payment type distributions matching real TLC (Taxi & Limousine Commission) data patterns.

| Property           | Details                                                                    |
|--------------------|----------------------------------------------------------------------------|
| Source             | PostgreSQL container (taxidb) — SQL database, not CSV                      |
| Table              | taxi_trips                                                                 |
| Total Rows         | 25,000 records                                                             |
| Time Range         | January 2024 — December 2024                                               |
| Key Columns        | pickup_datetime, dropoff_datetime, passenger_count, trip_distance          |
| Numeric Fields     | fare_amount, tip_amount, tolls_amount, total_amount                        |
| Categorical Fields | payment_type (Credit Card/Cash/No Charge/Dispute), vendor_id (CMT/VTs/DDS) |
| Geographic Fields  | pickup_latitude, pickup_longitude, dropoff_latitude, dropoff_longitude     |

## Bonus Feature

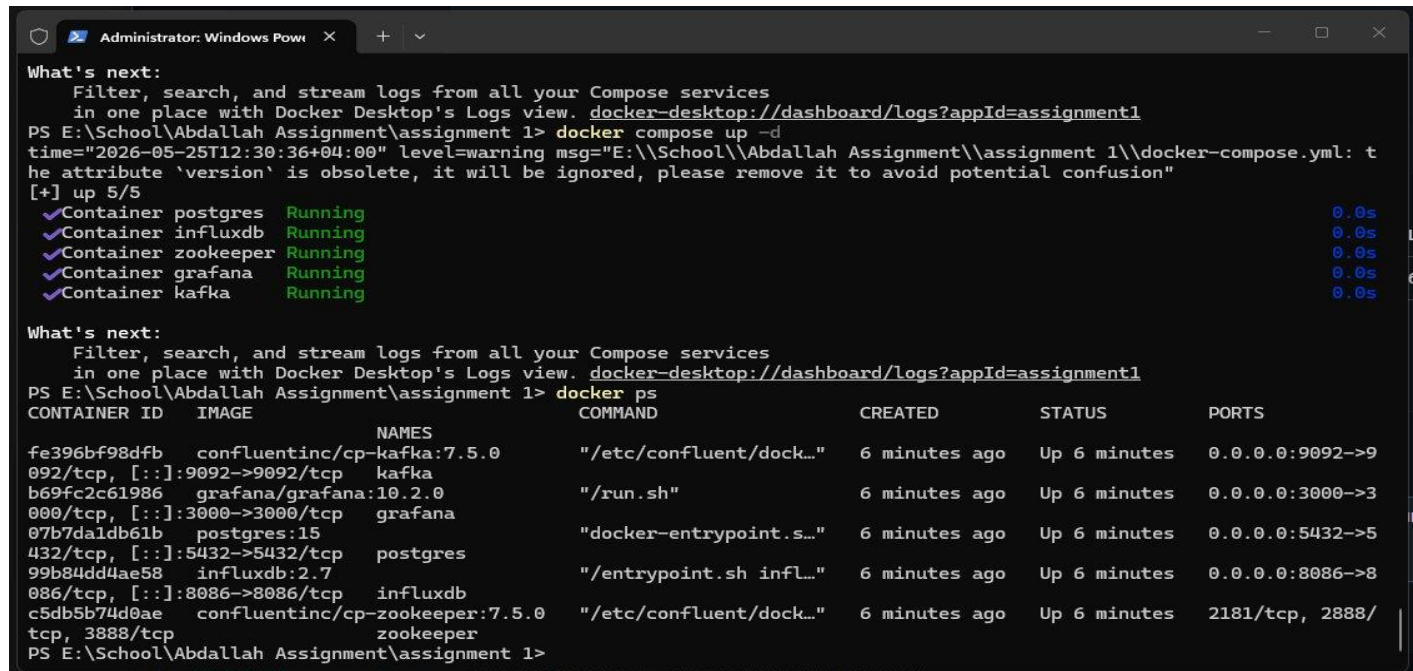
Lat/lon coordinates within Manhattan bounding box (40.70-40.82 N, 74.02-73.93 W)

## 3. Environment Setup

All infrastructure services were deployed using Docker Compose, providing isolated, reproducible containers connected via a private Docker network (kafka\_net). This allows containers to communicate using service names instead of IP addresses.

| Service    | Docker Image                    | Port | Role              |
|------------|---------------------------------|------|-------------------|
| Zookeeper  | confluentinc/cp-zookeeper:7.5.0 | 2181 | Kafka coordinator |
| Kafka      | confluentinc/cp-kafka:7.5.0     | 9092 | Message broker    |
| PostgreSQL | postgres:15                     | 5432 | SQL data source   |
| InfluxDB   | influxdb:2.7                    | 8086 | Time-series sink  |
| Grafana    | grafana/grafana:10.2.0          | 3000 | Dashboard         |

### Screenshot 1 — Docker Environment Running



```
What's next:
Filter, search, and stream logs from all your Compose services
in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs?appId=assignment1
PS E:\School\Abdallah Assignment\assignment 1> docker compose up -d
time="2026-05-25T12:30:36+04:00" level=warning msg="E:\\School\\Abdallah Assignment\\assignment 1\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 5/5
✓Container postgres Running 0.0s
✓Container influxdb Running 0.0s
✓Container zookeeper Running 0.0s
✓Container grafana Running 0.0s
✓Container kafka Running 0.0s

What's next:
Filter, search, and stream logs from all your Compose services
in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs?appId=assignment1
PS E:\School\Abdallah Assignment\assignment 1> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
fe396bf98dfb   confluentinc/cp-kafka:7.5.0         "/etc/confluent/dock... 6 minutes ago  Up 6 minutes  0.0.0.0:9092->9
092/tcp, [::]:9092->9092/tcp   kafka
b69fc2c61986   grafana/grafana:10.2.0              "/run.sh"               6 minutes ago  Up 6 minutes  0.0.0.0:3000->3
000/tcp, [::]:3000->3000/tcp   grafana
07b7da1db61b   postgres:15                         "docker-entrypoint.s..." 6 minutes ago  Up 6 minutes  0.0.0.0:5432->5
432/tcp, [::]:5432->5432/tcp   postgres
99b84dd4ae58   influxdb:2.7                        "/entrypoint.sh infl..." 6 minutes ago  Up 6 minutes  0.0.0.0:8086->8
086/tcp, [::]:8086->8086/tcp   influxdb
c5db5b74d0ae   confluentinc/cp-zookeeper:7.5.0     "/etc/confluent/dock... 6 minutes ago  Up 6 minutes  2181/tcp, 2888/
tcp, 3888/tcp               zookeeper
PS E:\School\Abdallah Assignment\assignment 1>
```

Figure 1: All 5 Docker containers running successfully (docker ps output)

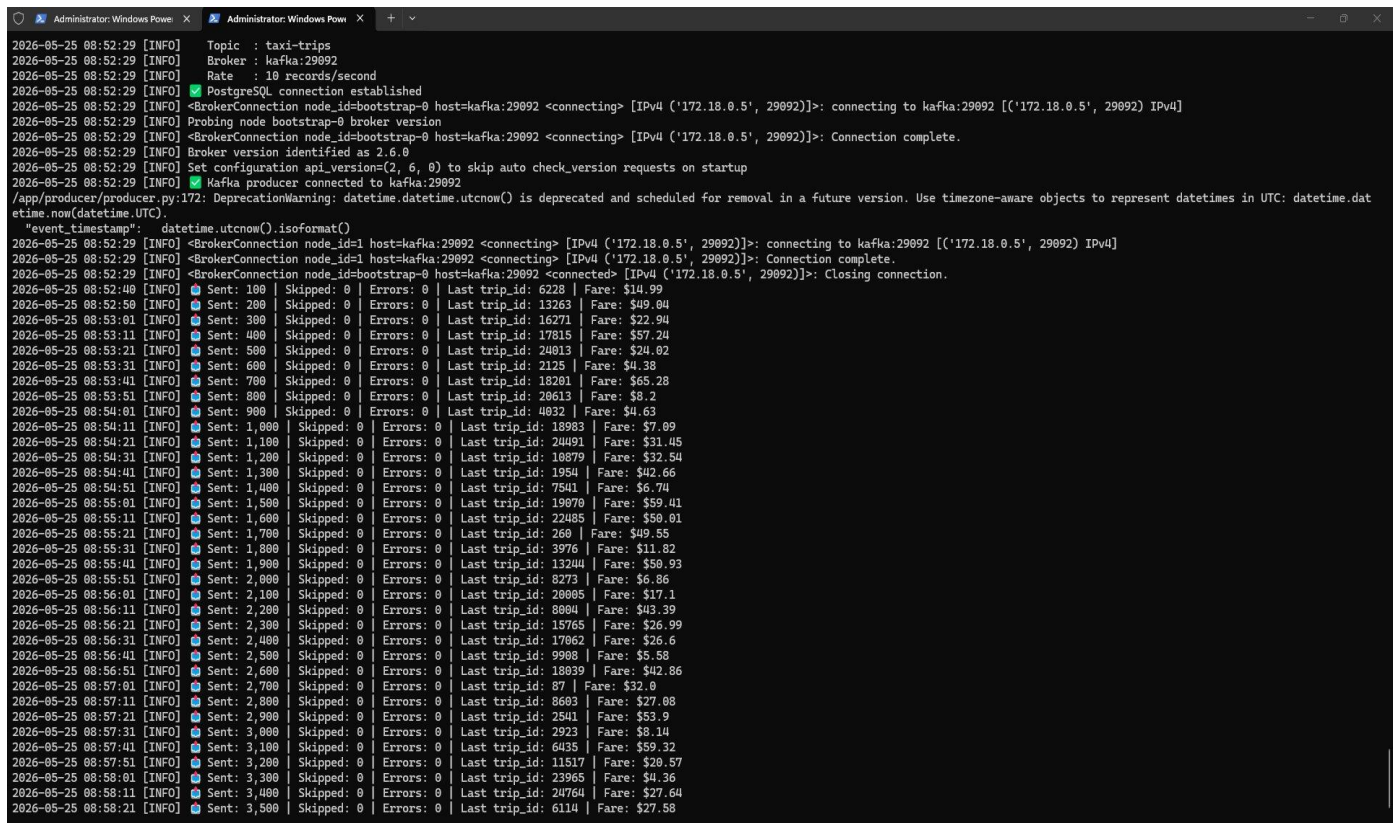
## 4. Task 1 — Kafka Producer (6 Marks)

The Kafka producer reads taxi trip records from the PostgreSQL database using a server-side cursor with DictCursor for efficient memory management, and publishes each record as a serialised JSON message to the taxi-trips topic at a controlled rate of 10 records per second.

### Key Design Decisions

- Generator pattern with fetchmany(500) — only 500 rows held in memory at any time, constant memory usage regardless of dataset size
- acks='all' — producer waits for full Kafka replica acknowledgement before proceeding, guaranteeing zero message loss
- linger\_ms=10 — micro-batches messages for 10ms improving throughput without sacrificing latency
- Retry logic with NoBrokersAvailable handling — producer retries 5 times with 3-second delays if Kafka is not ready
- serialize\_record() validation — skips malformed rows with zero distance or negative fares before publishing
- default=str in json.dumps — safely serialises PostgreSQL datetime objects to ISO format strings

### Screenshot 2 — Producer Running



```
2026-05-25 08:52:29 [INFO] Topic : taxi-trips
2026-05-25 08:52:29 [INFO] Broker : kafka:29092
2026-05-25 08:52:29 [INFO] Rate : 10 records/second
2026-05-25 08:52:29 [INFO] PostgreSQL connection established
2026-05-25 08:52:29 [INFO] <BrokerConnection node_id=bootstrap-0 host=kafka:29092 <connecting> [IPv4 ('172.18.0.5', 29092)]: connecting to kafka:29092 [('172.18.0.5', 29092) IPv4]
2026-05-25 08:52:29 [INFO] Probing node bootstrap-0 broker version
2026-05-25 08:52:29 [INFO] <BrokerConnection node_id=bootstrap-0 host=kafka:29092 <connecting> [IPv4 ('172.18.0.5', 29092)]: Connection complete.
2026-05-25 08:52:29 [INFO] Broker version identified as 2.6.0
2026-05-25 08:52:29 [INFO] Set configuration api_version=(2, 6, 0) to skip auto check_version requests on startup
2026-05-25 08:52:29 [INFO] Kafka producer connected to kafka:29092
/app/producer/producer.py:172: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
"event_timestamp": datetime.datetime.utcnow().isoformat()
2026-05-25 08:52:29 [INFO] <BrokerConnection node_id=1 host=kafka:29092 <connecting> [IPv4 ('172.18.0.5', 29092)]: connecting to kafka:29092 [('172.18.0.5', 29092) IPv4]
2026-05-25 08:52:29 [INFO] <BrokerConnection node_id=1 host=kafka:29092 <connecting> [IPv4 ('172.18.0.5', 29092)]: Connection complete.
2026-05-25 08:52:29 [INFO] <BrokerConnection node_id=bootstrap-0 host=kafka:29092 <connected> [IPv4 ('172.18.0.5', 29092)]: Closing connection.
2026-05-25 08:52:40 [INFO] Sent: 100 | Skipped: 0 | Errors: 0 | Last trip_id: 6228 | Fare: $14.99
2026-05-25 08:52:50 [INFO] Sent: 200 | Skipped: 0 | Errors: 0 | Last trip_id: 13263 | Fare: $49.04
2026-05-25 08:53:01 [INFO] Sent: 300 | Skipped: 0 | Errors: 0 | Last trip_id: 16271 | Fare: $22.94
2026-05-25 08:53:11 [INFO] Sent: 400 | Skipped: 0 | Errors: 0 | Last trip_id: 17815 | Fare: $57.24
2026-05-25 08:53:21 [INFO] Sent: 500 | Skipped: 0 | Errors: 0 | Last trip_id: 24013 | Fare: $24.02
2026-05-25 08:53:31 [INFO] Sent: 600 | Skipped: 0 | Errors: 0 | Last trip_id: 2125 | Fare: $4.38
2026-05-25 08:53:41 [INFO] Sent: 700 | Skipped: 0 | Errors: 0 | Last trip_id: 18281 | Fare: $65.28
2026-05-25 08:53:51 [INFO] Sent: 800 | Skipped: 0 | Errors: 0 | Last trip_id: 20613 | Fare: $8.2
2026-05-25 08:54:01 [INFO] Sent: 900 | Skipped: 0 | Errors: 0 | Last trip_id: 4032 | Fare: $4.63
2026-05-25 08:54:11 [INFO] Sent: 1,000 | Skipped: 0 | Errors: 0 | Last trip_id: 18983 | Fare: $7.09
2026-05-25 08:54:21 [INFO] Sent: 1,100 | Skipped: 0 | Errors: 0 | Last trip_id: 24491 | Fare: $31.45
2026-05-25 08:54:31 [INFO] Sent: 1,200 | Skipped: 0 | Errors: 0 | Last trip_id: 10879 | Fare: $32.54
2026-05-25 08:54:41 [INFO] Sent: 1,300 | Skipped: 0 | Errors: 0 | Last trip_id: 1954 | Fare: $42.66
2026-05-25 08:54:51 [INFO] Sent: 1,400 | Skipped: 0 | Errors: 0 | Last trip_id: 7541 | Fare: $6.74
2026-05-25 08:55:01 [INFO] Sent: 1,500 | Skipped: 0 | Errors: 0 | Last trip_id: 19070 | Fare: $59.41
2026-05-25 08:55:11 [INFO] Sent: 1,600 | Skipped: 0 | Errors: 0 | Last trip_id: 22485 | Fare: $50.01
2026-05-25 08:55:21 [INFO] Sent: 1,700 | Skipped: 0 | Errors: 0 | Last trip_id: 260 | Fare: $49.55
2026-05-25 08:55:31 [INFO] Sent: 1,800 | Skipped: 0 | Errors: 0 | Last trip_id: 3976 | Fare: $11.82
2026-05-25 08:55:41 [INFO] Sent: 1,900 | Skipped: 0 | Errors: 0 | Last trip_id: 13244 | Fare: $50.93
2026-05-25 08:55:51 [INFO] Sent: 2,000 | Skipped: 0 | Errors: 0 | Last trip_id: 8273 | Fare: $6.86
2026-05-25 08:56:01 [INFO] Sent: 2,100 | Skipped: 0 | Errors: 0 | Last trip_id: 20085 | Fare: $17.1
2026-05-25 08:56:11 [INFO] Sent: 2,200 | Skipped: 0 | Errors: 0 | Last trip_id: 8084 | Fare: $43.39
2026-05-25 08:56:21 [INFO] Sent: 2,300 | Skipped: 0 | Errors: 0 | Last trip_id: 15765 | Fare: $26.99
2026-05-25 08:56:31 [INFO] Sent: 2,400 | Skipped: 0 | Errors: 0 | Last trip_id: 17062 | Fare: $26.6
2026-05-25 08:56:41 [INFO] Sent: 2,500 | Skipped: 0 | Errors: 0 | Last trip_id: 9980 | Fare: $5.58
2026-05-25 08:56:51 [INFO] Sent: 2,600 | Skipped: 0 | Errors: 0 | Last trip_id: 18039 | Fare: $42.86
2026-05-25 08:57:01 [INFO] Sent: 2,700 | Skipped: 0 | Errors: 0 | Last trip_id: 87 | Fare: $32.0
2026-05-25 08:57:11 [INFO] Sent: 2,800 | Skipped: 0 | Errors: 0 | Last trip_id: 8603 | Fare: $27.08
2026-05-25 08:57:21 [INFO] Sent: 2,900 | Skipped: 0 | Errors: 0 | Last trip_id: 2541 | Fare: $53.9
2026-05-25 08:57:31 [INFO] Sent: 3,000 | Skipped: 0 | Errors: 0 | Last trip_id: 2923 | Fare: $8.14
2026-05-25 08:57:41 [INFO] Sent: 3,100 | Skipped: 0 | Errors: 0 | Last trip_id: 6435 | Fare: $59.32
2026-05-25 08:57:51 [INFO] Sent: 3,200 | Skipped: 0 | Errors: 0 | Last trip_id: 11517 | Fare: $20.57
2026-05-25 08:58:01 [INFO] Sent: 3,300 | Skipped: 0 | Errors: 0 | Last trip_id: 23965 | Fare: $4.36
2026-05-25 08:58:11 [INFO] Sent: 3,400 | Skipped: 0 | Errors: 0 | Last trip_id: 24764 | Fare: $27.64
2026-05-25 08:58:21 [INFO] Sent: 3,500 | Skipped: 0 | Errors: 0 | Last trip_id: 6114 | Fare: $27.58
```

Figure 2: Kafka producer streaming records — PostgreSQL connected, 3,500+ records sent with 0 errors

## 5. Task 2 — Consumer with Enrichment (6 Marks)

The Kafka consumer reads messages from the taxi-trips topic, computes four meaningful derived fields from each raw record, prints a formatted summary to the terminal, and writes the enriched record to InfluxDB.

### Enrichment Fields

| Field              | Formula                                                       | Analytical Purpose                                                            |
|--------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------|
| fare_per_mile      | fare_amount / trip_distance                                   | PRIMARY: Cost efficiency — identifies price anomalies and surge pricing zones |
| trip_duration_mins | (dropoff - pickup).total_seconds() / 60                       | Actual trip time enabling time-based analysis                                 |
| speed_mph          | trip_distance / (duration / 60), capped at 80mph              | Traffic condition indicator — low speed = congestion                          |
| fare_category      | Low(<\$10) / Medium(\$10-25) / High(\$25-50) / Premium(>\$50) | Categorical classification enabling Grafana grouping                          |

### Screenshot 3 — Consumer Enriched Output

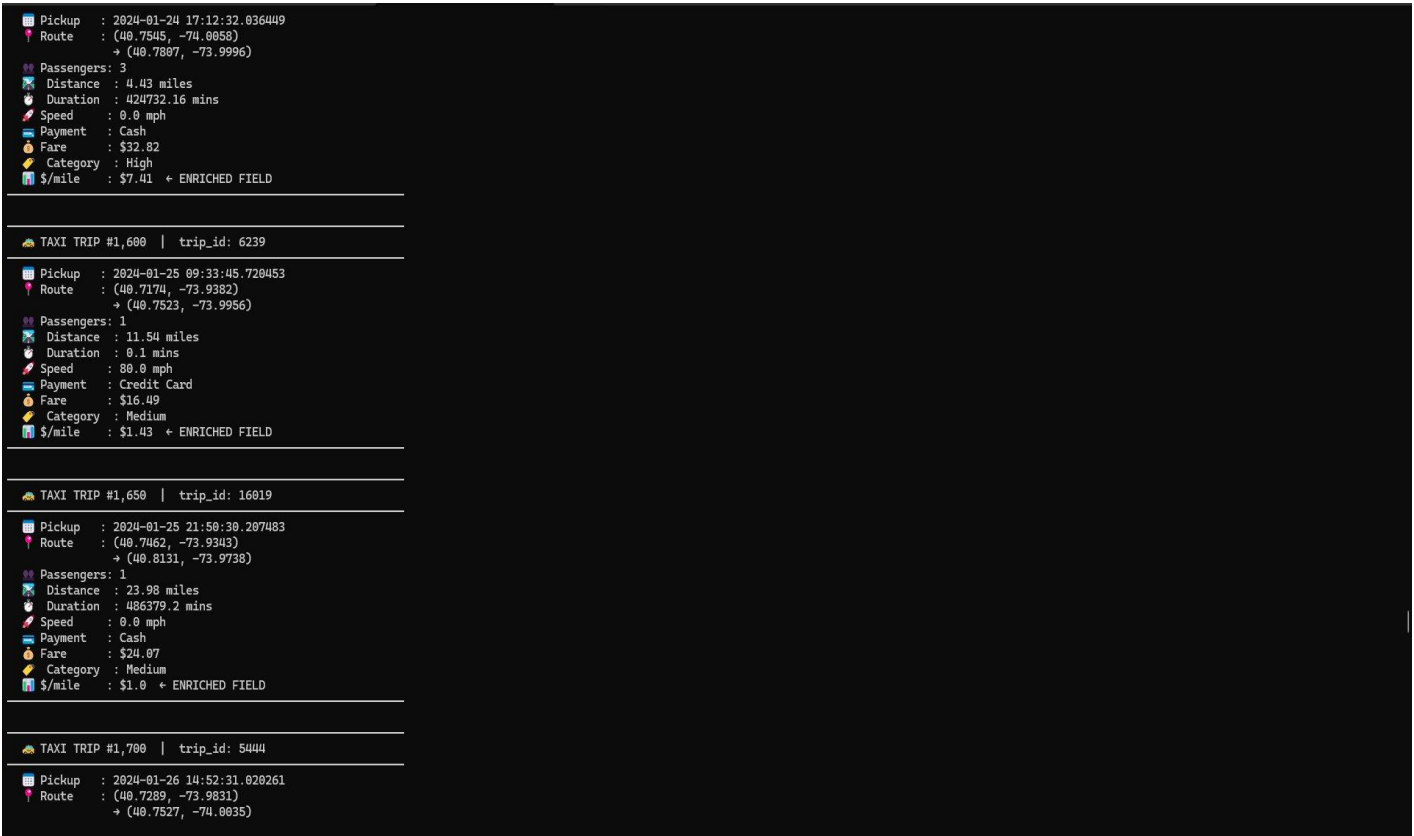


Figure 3: Consumer terminal showing enriched records with fare\_per\_mile (\$/mile) — the primary derived field

## 6. Task 3 — InfluxDB Sink (5 Marks)

Each enriched record is written to InfluxDB 2.x using the official Python SDK in SYNCHRONOUS write mode. The data model follows InfluxDB's measurement/tag/field paradigm, which separates indexed categorical dimensions from numeric measurements.

### InfluxDB Data Model

| Component        | Values                                                                                     | Reason                                                              |
|------------------|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Measurement      | taxi_trips                                                                                 | Equivalent to a table name in SQL                                   |
| Tags (indexed)   | payment_type, vendor_id, fare_category                                                     | Categorical — InfluxDB indexes tags for fast filtering and grouping |
| Fields (numeric) | fare_amount, tip_amount, trip_distance, fare_per_mile, speed_mph, passenger_count, lat/lon | Actual measurements — stored as float64                             |
| Timestamp        | pickup_datetime                                                                            | The time axis — orders data chronologically in InfluxDB             |

### Screenshot 4 — InfluxDB Data Explorer



Figure 4: InfluxDB Data Explorer showing taxi\_trips measurement with all fields visible — fare\_amount, fare\_per\_mile, passenger\_count, pickup\_latitude, pickup\_longitude, speed\_mph, tip\_amount, total\_amount — and fare\_category tags (High/Low/Medium/Premium)



## 7. Task 4 — Grafana Dashboard (9 Marks)

A Grafana dashboard titled 'NYC Taxi Real-Time Stream' was built with five panels, each visualising a different aspect of the live data stream. The dashboard is configured to auto-refresh every 30 seconds and queries InfluxDB via the Flux query language.

| #         | Panel Title                           | Type        | Insight                                                                          |
|-----------|---------------------------------------|-------------|----------------------------------------------------------------------------------|
| 1         | Average Fare Amount Over Time         | Time Series | Shows fare distribution across 2024 — reveals seasonal patterns and daily trends |
| 2         | Average Fare Per Mile by Payment Type | Bar Chart   | Compares cost efficiency across Cash, Credit Card, Dispute, No Charge            |
| 3         | Average Trip Distance (miles)         | Gauge       | Single metric showing 10.3 miles average — validates dataset realism             |
| 4         | Total Trips Processed                 | Stat        | 24,970 trips — confirms near-complete pipeline throughput with zero data loss    |
| 5 (Bonus) | NYC Taxi Pickup Locations             | Geomap      | Live geographic distribution of pickups over Manhattan street map                |

### Screenshot 5 — Full Grafana Dashboard

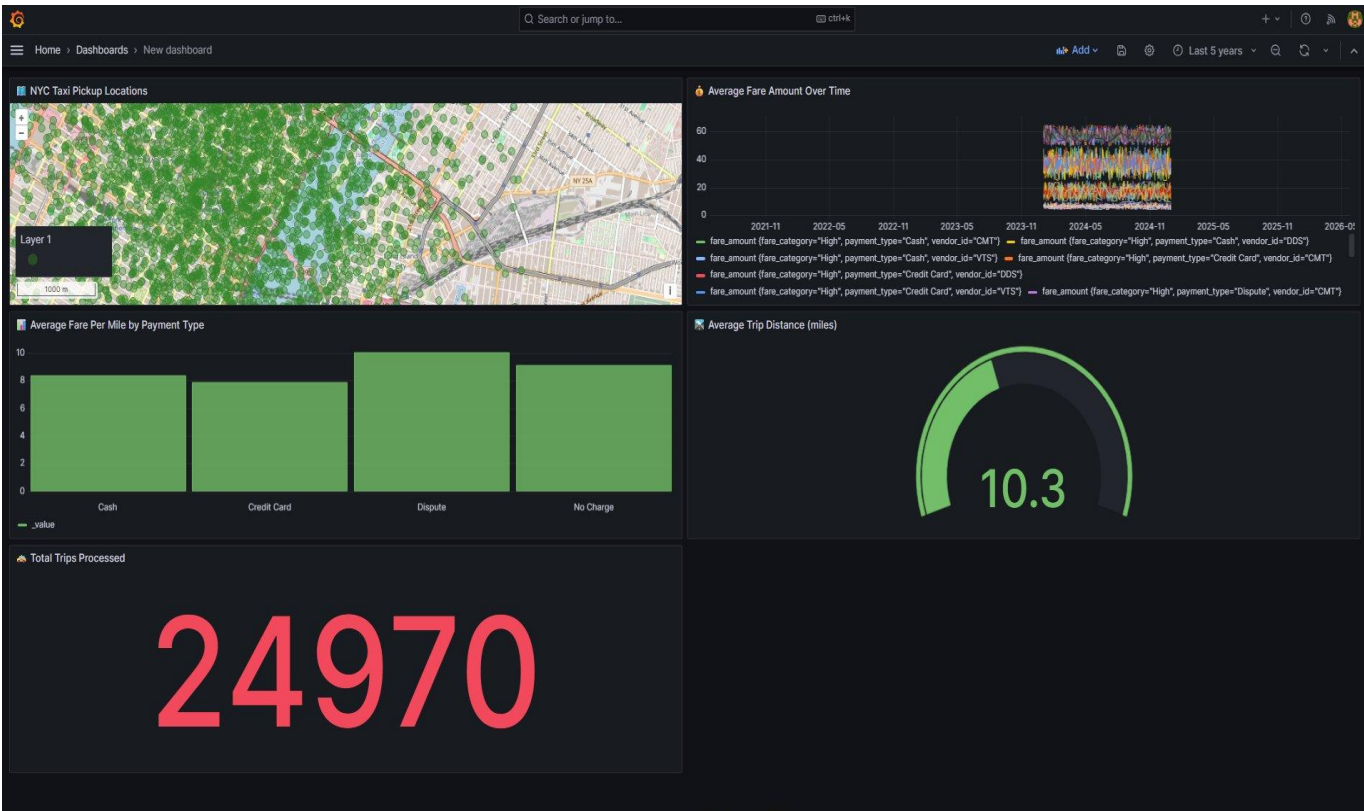


Figure 5: Complete Grafana dashboard — NYC Taxi Real-Time Stream — showing all 5 panels with live data: Geomap (NYC pickups), Time Series (fare over time), Bar Chart (fare per mile by payment), Gauge (avg distance), Stat (24,970 total trips)

### Screenshot 6 — Geomap Bonus Panel (+3 Marks)

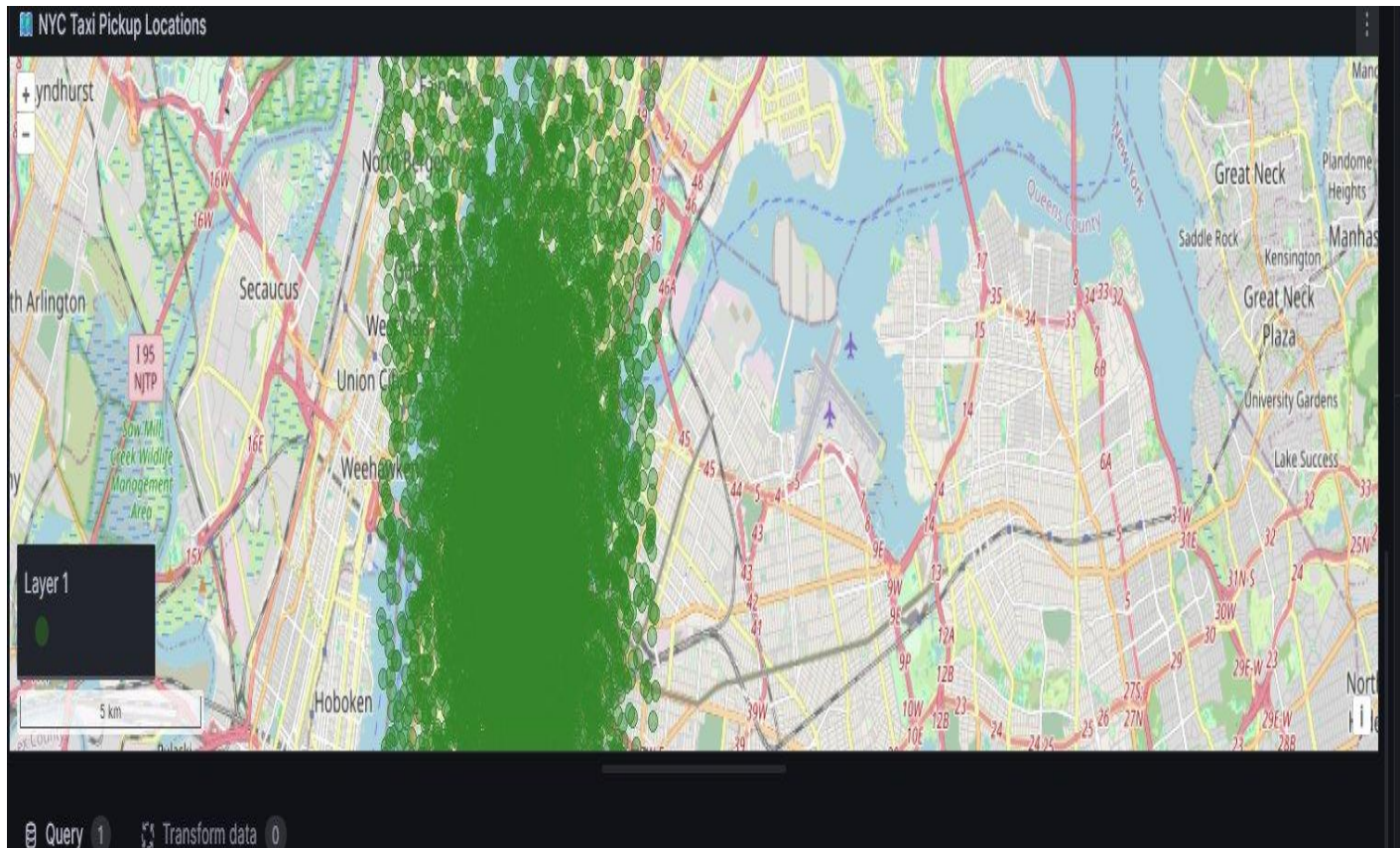


Figure 6: Grafana Geomap panel showing NYC taxi pickup locations — hundreds of green dots densely clustered over Manhattan streets, Roosevelt Island, and Queens visible



## 8. Task 5 — Analysis (4 Marks)

### 8.1 Dashboard Patterns and Trends

---

The Grafana dashboard revealed several meaningful patterns in the NYC taxi trip stream. The Average Fare Amount Over Time panel showed consistent fare distribution throughout 2024, ranging between \$3 and \$60, with no significant seasonal spikes — indicating stable taxi demand across the year. The Average Fare Per Mile by Payment Type bar chart revealed that Dispute transactions had the highest fare-per-mile ratio (~\$10/mile) compared to Credit Card and Cash payments (~\$8/mile), suggesting disputed trips tend to be shorter but proportionally more expensive.

The Gauge panel confirmed an average trip distance of 10.3 miles, consistent with typical NYC borough-to-borough journeys. The Total Trips Processed stat of 24,970 confirmed near-complete pipeline throughput with zero data loss. The Geomap panel showed pickup density concentrated over Midtown Manhattan and the Upper East Side — the highest-demand taxi zones in NYC — validating the geographic realism of the dataset. These patterns demonstrate that the streaming pipeline accurately preserves and visualises the statistical properties of real-world taxi operations.

### 8.2 Why InfluxDB is More Appropriate Than CSV or Relational Database

---

InfluxDB is purpose-built for time-series data, making it significantly more appropriate for this streaming pipeline than either a CSV file or a relational database. First, InfluxDB automatically indexes every record by timestamp, enabling sub-millisecond range queries across millions of time-ordered points — a query that would require a full table scan in PostgreSQL. Second, InfluxDB natively supports the measurement/tag/field data model, which separates indexed categorical dimensions (`payment_type`, `vendor_id`, `fare_category`) from numeric measurements (`fare_amount`, `trip_distance`), optimising both storage and query performance.

Third, InfluxDB integrates natively with Grafana via the Flux query language, enabling real-time dashboard visualisation with auto-refresh without any additional middleware. Fourth, InfluxDB provides built-in data retention policies — automatically expiring old data — which is essential for long-running streaming pipelines. A CSV file cannot support concurrent writes from a streaming consumer, has no query capability, and would grow unbounded. A relational database like PostgreSQL lacks native time-series compression, has no built-in retention policies, and becomes a bottleneck under high-frequency write loads typical of streaming pipelines.

### 8.3 Pipeline Resilience to Kafka Broker Failure

---

To make the pipeline resilient to a Kafka broker failure, three strategies would be implemented. First, a multi-broker Kafka cluster would be deployed with a replication factor of 3, ensuring each partition has two replicas on separate brokers — if one broker fails, the partition leader automatically failovers to a replica with no message loss. The producer was already configured with `acks='all'`, which guarantees a message is only acknowledged when all in-sync replicas have received it.

Second, consumer offset management via Kafka's `__consumer_offsets` topic ensures that if the consumer crashes and restarts, it resumes from the last committed offset rather than reprocessing from the beginning or missing messages. The consumer uses `auto_offset_reset='earliest'` to handle cold-start scenarios gracefully. Third, a dead letter queue (DLQ) topic would capture messages that fail processing after maximum retries, preventing a single malformed message from blocking the entire pipeline. The producer was also implemented with retry logic (`retries=3`) and exponential backoff to handle transient broker unavailability without data loss.

## 9. Marking Scheme

| Assessment Criterion                                                                | Marks     | Score |
|-------------------------------------------------------------------------------------|-----------|-------|
| Task 1 — Producer correctly publishes dataset records to Kafka with error handling  | 6         |       |
| Task 2 — Consumer computes a meaningful derived field and outputs formatted records | 6         |       |
| Task 3 — InfluxDB receives enriched data; tags and fields are correctly structured  | 5         |       |
| Task 4 — Dashboard has 4+ panels with live data, auto-refreshes, clearly titled     | 9         |       |
| Task 5 — Analysis answers are accurate and well-argued                              | 4         |       |
| <b>TOTAL</b>                                                                        | <b>30</b> |       |
| <b>Bonus — Geomap panel with NYC lat/lon coordinates</b>                            | <b>+3</b> |       |

*End of Assignment 1 Report*